

MOMONGA

DESKTOP COMPANION

GAME DESIGN DOCUMENT · v0.2



“밥. ...그리고 간식도.”

컴퓨터 안에서 제멋대로 살아가는 작은 동거인

Prototype scope: Momonga only | Platform: Windows 10/11 | Engine: C# / .NET 8 / WPF

※ 개발 프로토타입에서는 교체 가능한 플레이스홀더 아트를 사용한다. 실제 IP 캐릭터 아트/음성/로고 적용은 필요한 권리 범위에 맞춰 별도 적용한다.

0. 한 장으로 보는 게임

한 줄 정의

사용자가 컴퓨터를 쓰는 동안 바탕화면에서 제멋대로 먹고, 자고, 놀고, 간식을 요구하고, 빠지고, 다시 슬쩍 다가오는 자율형 데스크탑 펫.

항목	설계 방향
플레이 방식	앱을 "플레이"하기보다 평소 PC 사용 중 캐릭터와 함께 생활
핵심 감정	귀여움 + 뽀뽀함 + 뜻밖의 작은 사건
성장	Activity Point 로 밥·간식·장난감·침대·서식지 구매
AI	Needs + Emotion + Personality + Utility AI 로 자율행동
상호작용	쓰다듬기 / 찌르기 / 볼 당기기 / 딱밤 / 드래그 / 우클릭 원형 메뉴
집중 모드	캐릭터를 서식지에 넣고 "방해금지" 팻말을 세우는 물리적 시스템
프로토타입	모형가 1 종. 이후 데이터 기반으로 여러 캐릭터 확장

Design Pillars

살아있는 느낌 - 명령 대기형 UI 가 아니라 스스로 욕구를 해결하고 행동한다.
 거슬리지 않는 존재감 - 필요할 때만 다가오고, 사용자의 PC 작업을 망치지 않는다.
 성격이 먼저 보이는 시스템 - 같은 기능도 모형가답게 요구하고, 빠지고, 뽀뽀하게 반응한다.
 데스크탑 자체가 놀이터 - 커서, 서식지, 가구, 팻말 모두 실제 화면 위 오브젝트다.
 작은 사건의 누적 - 큰 퀘스트보다 "간식 줘 → 거절 → 빠짐 → 좀 뒤에 다시 옴" 같은 순간이 콘텐츠다.

1. 플레이어 판타지 & 핵심 루프

“컴퓨터를 켜더니 애도 자기 하루를 시작한다.”

플레이어가 느껴야 하는 것

내가 게임을 켜서 돌보는 것이 아니라, 내 컴퓨터에 애가 같이 산다.

방금 전 행동을 어느 정도 기억하고 있어서 같은 장난에도 반응이 달라진다.

밥과 간식, 가구를 사주는 이유가 단순 수치 회복이 아니라 새로운 행동을 보기 위해서다.

집중해야 할 때는 캐릭터를 직접 “집에 넣어두는” 감각으로 방해를 줄일 수 있다.

Core Loop

단계	행동	게임에서 생기는 것
1	평소처럼 키보드/마우스 사용	Activity Point 축적
2	캐릭터 자율생활 관찰	배고픔, 졸림, 심심함, 간식 욕구 등 사건 발생
3	돌보기 또는 장난치기	욕구/감정/관계값 변화
4	포인트 사용	음식, 간식, 장난감, 침대, 서식지 아이템 획득
5	새로운 생활패턴	선호 아이템, 새로운 대화와 애니메이션 발견
6	관계 축적	사용자 근처에서 자기, 반기기, 먼저 애정 요구 등 해금

2. 캐릭터: 프로토타입 모몽가

캐릭터 연기의 기준

“착한 반려동물”이 아니라, 귀엽다는 걸 아는 것처럼 행동하는 조금 뽀뽀한 동거인. 먹을 것과 관심에는 적극적이고, 싫은 건 표정과 행동으로 티낸다.

성향	Prototype 값	행동 해석
Clinginess	0.70	관심받고 싶을 때 먼저 접근하는 편
Playfulness	0.72	장난감·커서·직접 상호작용 선호
Timidity	0.28	사용자에게 쉽게 겁먹지는 않음
Mischief	0.78	영똥한 행동과 장난성 Idle 비중 높음
Patience	0.38	요구가 무시되면 금방 표정이 변함
FoodLove	0.78	밥 반응이 크고 식사 관련 행동이 자주 보임
SnackLove	0.94	간식 요청/기대 반응이 핵심 캐릭터성
Sleepiness	0.55	잠도 좋아하지만 먹는 것보다 우선순위 낮음

대표 Acting

간식 요구 → 사용자를 봄 → 간식 보관함/말풍선 → 다시 사용자를 봄 → “...간식.”

거절 → 잠깐 정지 → 뽀뽀 봄 → “진짜?” 같은 짧은 반응 → 등을 돌리거나 조금 떨어져 앓음.

딱밤 반복 → 당황 → 째려봄 → 손 막기 → 피하기 → 결국 빠져서 등지/빈백으로 감.

오래 방해금지 상태 → 말은 안 하지만 팻말 너머로 사용자만 슬쩍 쳐다봄.

기분이 풀린 뒤에는 거창한 화해 연출보다 아무 일 없었다는 듯 슬쩍 다시 옆에 오는 쪽이 귀엽다.

3. Needs, 감정, 관계

구분	값	역할
Needs	Hunger / Thirst / Energy / Social / Fun	짧은 주기로 변화하는 기본 생활 욕구
Emotion	Mood / Stress / Annoyance / Curiosity	최근 사건에 따라 행동 우선순위를 바꾸는 상태
Relationship	Affection / Trust	오랜 기간 천천히 변하는 사용자와의 관계
Intent	NeedFood / WantSnack / WantAttention / WantSleep 등	왜 사용자나 오브젝트에 접근하는지 명시

원칙

죽음 없음. 장기간 미접속도 “조금 배고프고 빠진 상태” 이상으로 과도하게 벌주지 않는다.
 Affection/Trust 는 가벼운 딱밤 몇 번으로 크게 떨어지지 않는다. 반복되는 패턴이 누적될 때만 천천히 반영한다.
 Need 수치는 HUD 에 상시 노출하지 않는다. 행동, 표정, 말풍선으로 먼저 읽히게 한다.
 디버그 패널에서만 모든 수치를 명확히 볼 수 있게 한다.

4. 자율 행동 AI

핵심

Random Pet 가 아니라 “지금 가장 하고 싶은 행동”을 매 순간 평가한다. 단, 너무 최적화된 로봇처럼 보이지 않도록 작은 랜덤성과 행동 지속시간을 둔다.

상황	올라가는 Utility	비고
Hunger 높음 + 밥 있음	Eat	가장 가까운 유효 Food Bowl 탐색
Hunger 높음 + 밥 없음	AskFood	사용자 접근 후 짧게 요구
SnackLove 높음 + 쿨다운 종료	AskSnack	배고픔과 별개로 “먹고 싶음” 이벤트
Energy 낮음	Sleep	선호 Bed / Nest / Beanbag 점수 가산
Fun 낮음	PlayToy / CursorPlay	장난감 보유 여부 반영
Social 낮음	ApproachUser / AskAttention	집중 상태와 쿨다운 고려
Annoyance 높음	Sulk / Avoid	사용자 접근 점수 감소
Quiet Habitat	Habitat-only behaviors	사용자에게 말을 거는 행동은 비활성

필수 Behavior State

Idle · LookAround · LookAtCursor · Wander · ApproachUser · AskAttention · AskFood · AskWater · AskSnack · Eat · Drink · Sit · Sleep · Play · Groom · Curious · Startled · Sulk · Hide · ReactToPetting · ReactToPoke · ReactToFlick · ReactToCheekPull · Dragged

5. Activity Point 경제

목표

포인트를 벌기 위해 게임을 하는 것이 아니라, 평소 PC 를 쓰다 보니 생활비가 쌓이는 구조. 연타·오토클릭 파밍은 의미 없게 만든다.

Activity 감지

키 입력 "내용"은 절대 기록하지 않는다. 입력 이벤트 발생 여부/활동량만 익명 집계한다.
 마우스 클릭, 이동, 스크롤 역시 좌표·대상 앱을 저장하지 않고 활동량 계산에만 사용한다.
 60 초 Activity Window 단위로 포인트를 계산한다. 단일 클릭/키 입력마다 직접 지급하지 않는다.
 AFK 상태에서는 AP 를 지급하지 않는다. 비정상적으로 높은 이벤트량도 분당/시간당 상한에 막힌다.

샘플 밸런스	AP
가벼운 활성 1 분	1 AP
일반적인 활성 1 분	2 AP
활발한 활성 1 분	최대 3 AP
시간당 Soft Cap	90 AP 전후
초기 지급	150 AP (튜토리얼용)

Prototype 가격 예시

카테고리	아이템	가격	역할
Food	기본 식사	20	포만감 높음
Food	견과류	30	선호도 + Mood
Food	과일	35	포만감/기분 균형
Snack	쿠키	25	Fun + Mood
Snack	작은 디저트	45	Affection 반응 큼
Snack	특별 간식	80	특수 애니메이션 후보
Toy	공	120	추격/밀기 행동
Toy	작은 인형	180	안고 자기/끌고 다니기
Rest	빈백	350	편안한 휴식
Rest	등지	500	선호 수면 행동
Focus	방해금지 팻말	Starter	핵심 UX 라 기본 지급

6. 음식 · 간식 · 요구

Food

Food는 Hunger 해결용. Inventory에서 밥그릇에 넣고, 캐릭터가 배고플 때 스스로 먹는다. 밥그릇이 비어있을 때만 사용자를 찾는다.

Snack

Snack은 Hunger보다 Mood / Fun / Affection에 더 큰 영향을 준다. “필요해서”가 아니라 “먹고 싶어서” 요구할 수 있다는 점이 중요하다.

단계	모몽가 행동
1. 욕구 발생	사용자를 한번 보고 간식 관련 Intent 생성
2. 관심 끌기	가까이 와서 쳐다보거나 주변을 서성임
3. 요구	“...간식.” / “하나만.” 같은 짧은 말풍선
4-A. 지급	종류에 따라 선호 반응 + Mood/Fun/Affection 변화
4-B. 거절	정지 → 뺨히 보기 → 짧은 불만 → 상황에 따라 빠짐
4-C. 무시	재촉은 최대 1 회. 이후 쿨다운 후 스스로 다른 행동

중요

간식을 많이 준다고 무조건 최적의 플레이가 되지 않게 한다. “응석 증가”는 Personality/최근 상호작용 Modifier로 표현하되, 플레이어를 처벌하는 시스템은 피한다.

7. Shop & Inventory

카테고리	예시	구매 후
Food	기본 식사 / 견과류 / 과일	수량형 소비품으로 Inventory 저장
Snack	쿠키 / 작은 디저트 / 특별 간식	수량형 소비품
Toy	공 / 인형 / 터널 등	소유 후 Habitat 배치 가능
Bed	방석 / 빈백 / 등지 / 해먹	Comfort 와 특별 수면 행동 제공
Habitat	밥그릇 / 물그릇 / 바닥매트 / 보관함	생활 AI 의 실제 목적지
Decoration	조명 / 쿠션 / 작은 장식	기본은 시각적, 일부 Mood modifier 가능

아이템 설계 원칙

비싼 물건 = 단순 숫자 버프가 아니라 새로운 행동/애니메이션을 더 많이 보여준다.
 가구는 전부 AI 목적지가 될 수 있어야 한다. “농기만 하는 스킨” 비중은 낮춘다.
 물 리필은 Prototype 에서 무료. 필수 생존재까지 경제 압박으로 만들지 않는다.
 Shop 은 큰 게임 상점이 아니라 작은 데스크탑 유틸리티 창처럼 가볍게 연다.

8. Habitat: 바탕화면 위의 집

컨셉

Desktop 자체가 세계이고, Habitat 은 그 위에 사용자가 꾸며주는 “집 구역”. 캐릭터와 가구는 작은 독립 투명 Window 로 존재한다.

Habitat Zone

프로토타입에서는 HabitatZone 을 하나 둔다. 편집 모드에서만 경계가 명확히 보이고, 평소에는 얇은 바닥매트/러그 정도만 보여도 된다. 가구는 이 Zone 과 연결되며, 캐릭터를 Zone 안에 직접 드래그해서 넣을 수 있다.

오브젝트	AI 사용
Food Bowl	배고프면 찾아가 식사
Water Bowl	목마르면 찾아가 물 마심
Beanbag	휴식 / 낮잠 / 뺨짐
Nest	주 수면 / 숨기
Ball	Fun 낮을 때 혼자 놀기
Small Doll	끌고 다니기 / 안고 자기
Do Not Disturb Sign	Habitat 안의 Pet 을 Quiet Habitat 상태로 전환

편집 모드

Habitat Edit 를 켜면 Simulation 을 제한하고, 오브젝트 Outline 을 표시한다. 이동 / 잠금 / 삭제 / 간단한 크기 변경을 지원한다. 모든 위치는 모니터 ID + 좌표로 저장한다.

9. 핵심 기능: 방해금지 팻말

“알림 끄기”가 아니라 “잠깐 여기서 놀고 있어.”

활성화 플로우

Step	사용자 행동	시스템
1	캐릭터를 마우스로 잡는다	Dragged 상태
2	Habitat Zone 안에 내려놓는다	PetInsideHabitat = true
3	방해금지 팻말을 Zone 위에 세운다	QuietSignActive = true
4	두 조건이 모두 참	QuietHabitat 상태 활성화
5	팻말 제거 또는 Pet 을 밖으로 꺼냄	즉시 해제

Quiet Habitat 에서 꺼지는 행동

AskAttention
 AskFood / AskWater / AskSnack
 IdleTalk / Greeting / 자발적 Speech Bubble
 Cursor Chase
 사용자에게 먼저 접근하기
 Attention Event

하지만 Simulation 은 계속된다

밥/물이 있으면 스스로 먹고 마신다.
 빈백/둥지에서 자거나 쉰다.
 장난감이 있으면 혼자 논다.
 Need 와 Emotion 은 계속 변한다.
 사용자를 방해하지 않지만 커서가 가까이 오면 눈으로만 슬쩍 볼 수 있다.

10. 방해금지의 스트레스 & 서식지 품질

중요한 밸런스

방해금지는 벌이 아니라 집중용 생활 기능. 짧게 넣어두는 건 거의 영향이 없고, 오래 반복될 때 Social/Fun 저하와 Stress 상승이 서서히 보인다.

조건	효과
짧은 Quiet Habitat	거의 영향 없음
중간 시간	Social 서서히 감소
장시간	Social/Fun 감소 + Stress 증가
Bed 존재	Stress 상승 속도 완화
Favorite Bed	추가 완화 + 특수 휴식 행동 가능
Toy 존재	Fun 저하 완화
Favorite Toy	혼자 노는 Utility 증가 + Stress 완화
Food/Water 충분	기본 Needs 자가 해결
현재 Annoyance 높음	오히려 혼자 있는 시간이 Mood 회복에 도움될 수 있음

말하지 않고 보여주는 Protest

팻말 너머로 사용자 뺨히 보기.

서식지 입구 근처에 앉아 있음.

팻말을 앞발로 톡톡 건드리거나 살짝 밀어보기.

축 늘어진 채 빈백에 퍼져 있기.

장난감을 갖고 놀다가도 가끔 사용자 쪽을 확인.

Prototype에서는 실제 탈출/팻말 파괴는 하지 않는다. 이후 캐릭터 Personality 별 특수행동 후보.

해제 직후

짧게 있었다면 평범하게 복귀. 오래 있었다면 Social/Stress에 따라 사용자에게 바로 접근하거나, 쓰다듬기를 요구하거나, 간식을 먼저 요구하거나, 잠깐 빠진 채 거리를 두는 식으로 차이를 만든다.

11. 직접 만지는 상호작용

Hit Region	가능 행동	반응 기준
Head	Pet / Forehead Flick	Trust, Stress, Annoyance
Face	Poke / Look reaction	Mood, Curiosity
Left/Right Cheek	Cheek Pull	당김 거리, 반복 횟수
Body	Pet / Poke / Drag	Affection, Trust
Tail	Touch / Startle	Timidity, 현재 상태

괴롭힘 반응 단계

1 회: “?” → 반복: 짚어봄 → 더 반복: 막기 → 계속: 피함 → 심함: 서식지/등지로 가서 빠짐. Annoyance 는 시간과 긍정 상호작용으로 자연 회복한다.

디자인 원칙

장난이 가능한 게임이어야 하지만 학대 시뮬레이션이 되어서는 안 된다. 반응은 과장된 만화적 불쾌함과 회피 중심이며, 영구적 상처/죽음 같은 시스템은 두지 않는다.

12. 우클릭 원형 메뉴

카테고리	Prototype 행동
애정	쓰다듬기 / 칭찬 / 턱 긁기
돌보기	밥 주기 / 간식 주기 / 물 채우기 / 상태 보기
놀이	놀아주기 / 장난감 사용 / 커서 쫓기
장난	콧 피르기 / 볼 당기기 / 딱밤
대화	이름 부르기 / 뭐해? / 배고파? / 졸려?
생활	이리 와 / 밥 먹어 / 물 마셔 / 자러 가
기타	가만히 있기 / 위치 고정 / 설정

Context Sensitive 예시

Sleep: 깨우기 / 조용히 쓰다듬기 / 이불 덮어주기 / 딱밤 | Sulk: 달래기 / 간식 주기 / 쓰다듬기 / 또 놀리기 | Eating: 구경 / 칭찬 / 음식 더 주기

13. 커서와 사용자 Activity 인식

Cursor

가까운 커서를 눈/머리로 추적.
빠르게 지나가면 움찔.
Fun 이 낮거나 장난기 높은 상태면 가끔 따라가거나 잡으려 함.
실제 OS 포인터를 강제로 움직이지 않는다.

User Activity

Active / Idle / LongActiveSession 정도만 판별.
LongActiveSession 에서는 Social 기반 관심 요구 Utility 를 조금 올릴 수 있다.
키 내용, 화면 내용, 문서, 브라우저 히스토리, 앱 내용은 읽지 않는다.
전체화면 게임/영상 감지 시 안전하게 자발적 말풍선과 사용자 접근을 줄이는 "OS Courtesy"는 유지할 수 있다. 이것은 방해 금지 팻말을 대체하는 수동 Focus 기능이 아니라 시스템 차원의 방해 최소화다.

14. 대사 & 말풍선

Tag	톤 예시
Hungry	"밥." / "그릇 비었어."
Thirsty	"물 없어."
SnackRequest	"...간식." / "하나만."
Attention	"뭐 해?" / "나도 봐."
Annoyed	"야." / "...또?" / "진짜?"
Sleepy	"졸려." / "먼저 잘게."
ReturnGreeting	사용자 활동 재개 시 짧은 반가움
Quiet Habitat	기본적으로 Speech Bubble 없음. 표정과 행동으로만 표현

반복 방지

최근 Dialogue History 를 저장하고 같은 문장을 연속 재생하지 않는다. 한 상태당 여러 개의 짧은 Template 을 두고 Personality/Relationship 에 따라 가중치를 다르게 준다.

15. 관계 성장

관계 상태	행동 변화
초기	조금 거리 둠 / 만짐에 살짝 경계 / 혼자 쉬는 비중 높음
Affection 상승	사용자 근처에 머무름 / 먼저 접근 / 간식·관심 요구 증가
Trust 상승	쓰다듬기 허용 / 가까이서 잠 / 드래그·장난 반응이 덜 경계적
높은 관계	사용자 근처 수면 / 반기기 / 먼저 기대기 / 특별 Idle
Annoyance 누적	단기적으로 회피 / 손 막기 / 빼짐. 장기 관계값과 분리

중요

Affection 과 Trust 는 “정답 플레이” 점수가 아니다. 사용자가 장난도 치고, 가끔 무시도 하면서 각자 다른 관계 리듬을 만드는 쪽이 목표.

16. 장기 진행 & 수집

AP로 소비품을 사는 단기 루프와, 가구/서식지를 하나씩 늘리는 중기 목표를 분리한다.

비싼 침대/장난감은 효율보다 “새 행동을 보기 위한 콘텐츠”로 만든다.

특정 관계값, 누적 사용시간, 행동 발견으로 특별 대사/Idle/아이템을 해금할 수 있다.

업적은 과도한 체크리스트보다 “처음으로 간식 거절 후 빼짐 보기”, “방해금지 팻말 앞에서 항의 행동 보기” 같은 생활 사건 중심이 어울린다.

추천 진행 속도

기본 소비품은 10~30 분 사용으로 충분히 구매 가능하게, 장난감은 1~3 시간, 좋은 휴식 가구는 4~8 시간 정도의 누적 사용 목표가 되도록 시작한 뒤 플레이테스트로 조정한다.

17. 멀티 캐릭터 확장 설계

프로토타입은 모뎡가만

엔진은 처음부터 CharacterDefinition 기반으로 설계한다. 이후 여러 캐릭터가 추가되어도 외형만 바뀌는 스킨이 아니라 성격·선호·대사·행동 가중치가 달라져야 한다.

CharacterDefinition	내용
Identity	CharacterId / DisplayName / AssetSet
Personality	Clinginess / Playfulness / Timidity / Mischief / Patience / FoodLove / SnackLove / Sleepiness
Preferences	FavoriteFoods / FavoriteSnacks / FavoriteToys / PreferredBeds
Content	DialogueSet / AnimationSet / SoundSet
AI	BehaviorModifiers / RelationshipModifiers / SpecialBehaviors

향후 동시 생활

초기 확장에서는 Active Companion 1 명을 기본으로 하고, 이후 2~3 명 동시 생활을 옵션으로 고려한다. 캐릭터끼리 같이 먹기, 장난감 뺏기, 옆에서 자기, 따라다니기 등은 별도 Milestone 으로 분리한다.

18. Persistence & Offline

저장 항목	예시
Character	선택 캐릭터 / Personality data version
State	Needs / Mood / Stress / Annoyance
Relationship	Affection / Trust
Economy	AP / 구매 기록 최소 정보
Inventory	Consumable quantity / Furniture ownership
Habitat	Object ID / Position / Scale / Rotation / Locked / Zone membership
Bowls	현재 음식 / 잔량 / 물 잔량
History	최근 대사 / 최근 상호작용 / 쿨다운
System	Settings / LastSaveTimestamp

Offline Progress

종료 시간을 저장하고 재실행 시 경과 시간을 계산하되 Need 변화에 상한을 둔다. 오랫동안 접속하지 않았다고 죽거나 복구 불가능한 상태가 되지 않는다.

19. Windows 구현 방향

영역	권장 구조
Framework	C# / .NET 8 / WPF
Rendering	작은 Transparent Borderless Window 들
Pet	PetWindow 가 실제로 화면 좌표 이동
Habitat	HabitatZoneWindow + HabitatItemWindow
UI	SpeechBubbleWindow / RadialMenuWindow / Shop / Inventory
AI	Simulation 과 UI 분리, UtilityBehavior 인터페이스
Platform	CursorTracker / UserActivityMonitor / FullscreenDetector / TrayService
Storage	Prototype 은 JSON + Atomic Save
Threading	Simulation tick 과 UI Dispatcher 경계 명확히

피해야 할 것

화면 전체를 덮는 거대한 투명 Overlay Window. 빈 공간까지 클릭을 가로채면 데스크탑 컴패니언의 핵심 UX 가 깨진다.

20. Prototype v0.1 Scope

Milestone	완료 기준
M1 - Desktop Base	투명 PetWindow / Drag / Cursor look / Idle-Wander / Tray
M2 - Life	GameClock / Needs / Utility AI / Debug panel
M3 - Habitat	Habitat Zone / Food Bowl / Water Bowl / Beanbag / 이동
M4 - Economy	Activity Point / Wallet / Shop / Inventory / Food
M5 - Snack	간식 Inventory / AskSnack / 선호 반응 / Speech Bubble
M6 - Touch	Radial Menu / Pet / Poke / Flick / Cheek Pull
M7 - Emotion	Annoyance / Sulk / Affection / Trust / Personality Modifier
M8 - Quiet Habitat	DND Sign / Habitat confinement / Stress trade-off / visual protest
M9 - Polish	Save / Offline / Settings / Fullscreen courtesy / sound hooks

첫 20 분 경험

등장 → 커서 관찰 → 혼자 돌아다님.

사용자가 평소처럼 작업 → AP 자연 축적.

배고파짐 → 빈 밥그릇 확인 → 사용자에게 와서 "밥."

Shop 에서 음식 구매 → 그릇 채움 → 스스로 먹음.

조금 뒤 슬쩍 와서 "...간식."

쿠키를 주면 과장된 기쁨. 이후 볼 당기기/딱밤으로 Annoyance 반응 확인.

집중할 일이 생기면 캐릭터를 Habitat 에 넣고 DND Sign 설치 → 조용해짐.

한참 뒤 꺼내면 현재 Social/Stress 에 맞는 재회 반응.

21. Debug & Balancing

Debug HUD

Hunger · Thirst · Energy · Social · Fun · Mood · Stress · Annoyance · Affection · Trust · Current Behavior · Current Intent · Utility Scores · AP · Activity State · QuietHabitat State

강제 테스트

Set Hunger/Thirst/Energy
Add AP
Force Behavior
Empty/Fill Food Bowl
Empty/Fill Water Bowl
Force Snack Request
Set Annoyance
Toggle PetInsideHabitat / QuietSignActive
Advance Time 10m/1h
Corrupt-save recovery test

초기 밸런스 체크 질문

1 시간 평범하게 작업했을 때 밥/간식 구매가 너무 빠듯하거나 넘치지 않는가?
간식 요청이 귀엽기 전에 귀찮아지지 않는가?
방해금지 30~60 분은 부담 없이 쓸 수 있는가?
좋은 침대/장난감이 Quiet Habitat 의 스트레스 완화를 체감 가능하게 만드는가?
같은 반응이 30 분 안에 반복되어 "스크립트"처럼 보이지 않는가?

22. 제품 원칙

사용자 입력의 실제 내용은 기록하지 않는다.

캐릭터는 죽지 않는다.

알림 스팸 앱이 되지 않는다.

포인트를 위해 클릭을 반복시키지 않는다.

사용자의 실제 마우스 포인터를 납치하지 않는다.

방해금지 기능은 설정 스위치가 아니라 "Habitat + 팻말 + 캐릭터 배치"라는 물리적 플레이로 구현한다.

캐릭터의 성격이 시스템 위에 얹히는 것이 아니라, 시스템 선택 확률과 반응의 중심이 된다.

콘텐츠 볼륨보다 "얘가 진짜 사는 것 같다"는 느낌을 먼저 완성한다.

"작은 행동 하나가 귀여우면, 기능 열 개보다 오래 켜둔다."

APPENDIX A. Codex 시작 프롬프트

아래 프롬프트는 새 프로젝트 폴더에서 Codex 에 첫 지시로 넣는 용도다. GDD 파일을 프로젝트 루트에 함께 두고 사용한다.

You are the lead engineer for a Windows desktop virtual-pet project.

PROJECT

Build a functional prototype of “MOMONGA DESKTOP COMPANION”, a Windows desktop companion / Tamagotchi-style life simulation. The prototype contains one character only. Use replaceable placeholder art; do not spend development time recreating production IP artwork. The gameplay architecture must support additional character definitions later.

SOURCE OF TRUTH

Read GDD.pdf (or GDD.md if provided) in the project root before coding. Treat the GDD as the product source of truth. If implementation choices are not specified, choose the simplest architecture that preserves the intended user experience. Do not silently replace physical in-world interactions with ordinary settings toggles.

TECH STACK

- C#
- .NET 8
- WPF
- Windows 10/11
- JSON persistence for prototype

PRODUCT FEEL

The pet is not a widget waiting for commands. It should independently wander, look at the cursor, eat, drink, sleep, play, ask for attention, ask for food or snacks, become annoyed, sulk, recover, and use furniture. The target feeling is “a tiny creature lives on my desktop.”

WINDOW ARCHITECTURE

Do NOT create one fullscreen transparent overlay that intercepts the desktop.

Use small independent transparent borderless windows:

- PetWindow
- HabitatZoneWindow
- HabitatItemWindow
- SpeechBubbleWindow
- RadialMenuWindow

Only visible interactive entities should receive pointer input. Empty desktop areas must remain fully usable.

CORE MODULES

Keep Simulation, UI, Platform, Persistence and Content data separated.

Suggested modules:

```
/Core: AppController, GameClock, EventBus
/Character: CharacterDefinition, PersonalityProfile, CharacterPreferences
/Simulation: PetState, NeedSystem, EmotionState, RelationshipSystem
/AI: BehaviorBrain, UtilityBehavior, IntentSystem, BehaviorContext
/Behaviors: Idle, Wander, LookAtCursor, Eat, Drink, Sleep, Play, AskFood, AskSnack, AskAttention, Sulk, Hide
/Economy: ActivityPointService, Wallet, ShopService
/Inventory: InventoryService, ItemDefinition
/Habitat: HabitatManager, HabitatZone, FoodBowl, WaterBowl, Bed, Beanbag, Nest, Toy, DoNotDisturbSign
/UI: PetWindow, SpeechBubbleWindow, RadialMenuWindow, ShopWindow, InventoryWindow, SettingsWindow, DebugWindow
/Input: CursorTracker, PetInteractionDetector, UserActivityMonitor
/Persistence: SaveData, SaveService
/Animation: PetAnimator, AnimationDefinition
/Platform: StartupService, FullscreenDetector, MonitorService, TrayService
```

CHARACTER DATA

Do not hardcode personality directly into behavior classes. Create CharacterDefinition data with:

CharacterId, DisplayName, AssetSet, DialogueSetId, AnimationSetId, Clinginess, Playfulness, Timidity, Mischief, Patience, FoodLove, SnackLove, Sleepiness, FavoriteFoods, FavoriteSnacks, FavoriteToys, PreferredBeds, BehaviorModifiers, RelationshipModifiers.

Prototype personality: food-motivated, extremely snack-motivated, playful, attention-seeking, slightly shameless, expressive when refused, easily annoyed by repeated teasing, but quick to recover.

NEEDS AND EMOTIONS

Needs 0..100:

Hunger, Thirst, Energy, Social, Fun.

Emotion/state:

Mood, Stress, Annoyance, Curiosity.

Long-term relationship:

Affection, Trust.

Pet never dies. Long absence must not produce extreme punishment.

UTILITY AI

Each behavior should expose equivalents of CanExecute(context), EvaluateUtility(context), Start(), Update(), Stop(). Add minimum behavior duration, cooldown, and small score randomness.

Examples:

- Hunger high + food exists => Eat rises.
- Hunger high + bowl empty => AskFood rises.
- Snack cooldown ended + SnackLove high => AskSnack can rise.
- Energy low => Sleep rises, preferred beds score higher.
- Fun low => Toy or cursor play rises.
- Social low => user approach rises unless Quiet Habitat is active.
- Annoyance high => Sulk/Avoid rises.

INTENT

Track why the pet is acting: NeedFood, NeedWater, WantSnack, WantAttention, WantPlay, WantSleep, WantPrivacy. Use intent to drive animation and dialogue.

ACTIVITY POINTS

Earn points from normal computer use. PRIVACY IS NON-NEGOTIABLE:

- never record typed text
- never store pressed key identities
- never inspect documents, browser history or screen contents
- only aggregate anonymous activity signals such as "keyboard activity occurred", mouse activity, scroll activity and active time

Use activity windows, not 1 point per event. Start with configurable values around 1-3 AP per active minute and an hourly soft cap around 90 AP. AFK earns nothing. Input spam must not generate unlimited points.

SHOP / INVENTORY

Categories: Food, Snack, Toy, Bed, Habitat.

Starter examples:

Food: Basic Meal, Nuts, Fruit

Snack: Cookie, Small Dessert, Special Snack

Toy: Ball, Small Doll

Rest: Basic Cushion, Beanbag, Nest

Habitat: Food Bowl, Water Bowl, Do Not Disturb Sign

Consumables have quantity. Furniture is ownership + placed instances.

Water refill is free in prototype.

SNACK REQUEST

Snacks are not just hunger recovery. They mainly affect Mood/Fun/Affection. Implement AskSnackBehavior with cooldown, SnackLove, recent snack history and current mood as inputs.

Expected acting: approach user -> look up -> speech bubble "...간식." -> if ignored, one mild retry at most -> if refused, stare / small annoyed reaction / walk away or sulk based on state.

HABITAT

Create a draggable/resizable HabitatZone. In normal mode it can look like a subtle mat/area; in edit mode show clear bounds. Habitat items belong to a zone and persist monitor ID + position + scale + rotation + lock state.

Pet can be physically dragged into the zone.

CRITICAL FEATURE: QUIET HABITAT / DO NOT DISTURB SIGN

Do NOT implement the main Do Not Disturb feature as a simple settings toggle.

It is a physical gameplay interaction.

Activation conditions:

1. Pet is physically inside HabitatZone.
2. DoNotDisturbSign is placed/active on that same HabitatZone.

When both are true => QuietHabitat = true.

Removing the sign OR dragging the pet outside immediately disables it.

While QuietHabitat is active, disable autonomous user-facing interruption behaviors:

- AskAttention
- AskFood
- AskWater
- AskSnack
- IdleTalk
- Greeting
- autonomous SpeechBubble
- CursorChase
- autonomous approach to user

BUT DO NOT PAUSE SIMULATION.

Inside the habitat the pet should still:

- eat if food exists
- drink if water exists
- sleep/rest
- use toys
- wander locally
- look toward the user/cursor silently

Quiet Habitat trade-off:

- short duration: almost no penalty
- longer duration: Social decreases
- long duration: Social/Fun decrease and Stress slowly rises
- Bed reduces Stress growth
- Favorite Bed reduces it more
- Toys protect Fun and may reduce Stress
- Food/Water allow self-care
- if Annoyance is already high, private time may even help Mood recover

No popup should break Quiet Habitat just to warn the user.

Show discomfort visually instead:

- stare from behind the sign
- sit near habitat edge
- tap/push the sign
- flop on beanbag
- periodically glance toward user

Prototype must NOT automatically escape or destroy the sign. That can become a later personality-specific behavior.

When Quiet Habitat ends, choose a reunion response from current state:

- normal return
- immediately ask for affection
- ask for snack if eligible
- briefly remain sulky if Stress is high

DIRECT INTERACTION

Hit regions: Head, Face, LeftCheek, RightCheek, Body, Tail.

Implement: Pet, Poke, CheekPull, ForeheadFlick, Drag.

Petting = slow repeated drag over head/body.

CheekPull = hold cheek and drag outward; placeholder stretch is acceptable.

ForeheadFlick can start as Radial Menu > Tease > Forehead Flick, then next forehead click triggers reaction.

Repeated teasing raises Annoyance through escalating reactions: confused -> annoyed stare -> block -> avoid -> sulk.

Annoyance decays and can recover through time, petting, food, snacks or play.

RADIAL MENU

Right-click pet opens Sims-like radial categories:

Affection / Care / Play / Tease / Talk / Lifestyle / More.

Make options context-sensitive. Sleeping and sulking states should expose different actions.

DIALOGUE

Keep dialogue data outside code. Tags: Hungry, Thirsty, SnackRequest, Happy, Annoyed, Sleepy, Greeting, ReturnGreeting, PetReaction, FlickReaction, IdleTalk. Keep recent dialogue history to avoid immediate repetition. Quiet Habitat should normally have NO autonomous speech bubbles.

CURSOR / ACTIVITY

Pet watches nearby cursor and may startle at fast motion. It can chase visually but must never force-move the real OS pointer.

User activity states can be Active / Idle / LongActiveSession. A fullscreen detector may suppress spontaneous interruptions as basic OS courtesy, but this is NOT a replacement for the physical Do Not Disturb Sign mechanic.

PERSISTENCE

Persist PetState, relationship, AP, inventory, habitat items and positions, bowl contents, recent dialogue/interactions, settings and LastSaveTimestamp. Use safe/atomic JSON saving where practical. Corrupt save must not prevent app boot; back it up and recover defaults.

DEBUG WINDOW

Show all Needs, Mood, Stress, Annoyance, Affection, Trust, current Behavior, Intent, utility scores, AP, user activity state, PetInsideHabitat, QuietSignActive and QuietHabitat.

Provide test controls: set needs, add AP, force behavior, empty/fill bowls, force snack request, set annoyance, toggle habitat conditions, advance simulated time.

DEVELOPMENT ORDER

Do not attempt the whole product in one pass.

M1: WPF boots, transparent PetWindow, drag, cursor tracking, idle/wander, tray.

M2: GameClock, PetState, Needs, Utility AI, Debug Window.

M3: HabitatZone + Food Bowl + Water Bowl + Beanbag + movement to targets.

M4: Activity Points + Wallet + Shop + Inventory + Food consumption.

M5: Snack system + AskSnack + Speech Bubble.

M6: Radial Menu + Pet/Poke/Flick/CheekPull.

M7: Relationship + Annoyance + Sulk + Personality modifiers.

M8: Do Not Disturb Sign + Quiet Habitat + Stress trade-off + silent protest behaviors.

M9: Save/load + offline progress + settings + fullscreen courtesy + polish hooks.

AFTER EACH MILESTONE

- build the solution
- run relevant tests
- fix compile errors
- fix obvious runtime errors
- keep the repository runnable
- update README "Current Milestone / Controls / Known Limitations"

FIRST-RUN CONTENT

Starter habitat: Food Bowl, Water Bowl, Basic Cushion or Beanbag, Do Not Disturb Sign.

Starter inventory: Basic Meal x2, Cookie x1.

Starter AP: about 150.

FIRST EXPERIENCE TARGET

The vertical slice should allow this full story:

pet appears -> watches cursor -> wanders -> user works -> AP increases -> pet gets hungry -> checks empty bowl -> approaches and says "밥." -> user buys food -> fills bowl -> pet eats -> later asks "...간식." -> user gives cookie -> happy reaction -> user pulls cheek/flicks -> pet gets annoyed and sulks -> user needs focus -> drags pet into habitat and places DND sign -> pet stops interrupting but continues living inside -> after enough time silently stares/taps sign as Social/Stress changes -> user removes sign or pulls pet out -> pet reacts based on current state.

MOST IMPORTANT PRODUCT RULE

Prioritize believable small life events over feature count. The pet must be entertaining without demanding constant attention. Build the feeling that it lives on the desktop before adding content volume.